

Coding & Solution

Date: 30 September 2026

Team ID	6abc95cbdfc814753aa5fe42
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks
Project Type	AI-powered Learning Assistant Web Application

Coding & Solution

This section presents the implementation approach, technology stack, core modules, setup procedure, and code-quality practices of EduGenie. The application uses Google Gemini to provide AI-assisted learning features through a web-based interface.

Solution Summary

Field	Details
Repository Link / URL	Live application / public GitHub repository link: [Paste your link here]
Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	FastAPI, Jinja2 templates, Uvicorn
AI Technology	Google Gemini API
Frontend	HTML, CSS and Jinja2-based web pages
Backend	Python FastAPI

Key Features Implemented

- **AI Question & Answer:** Users can ask academic questions and receive Gemini-powered responses.
- **Concept Explanation:** Complex topics can be explained in a learner-friendly manner.
- **Quiz Generation:** The application can generate quizzes based on the requested topic.
- **Text Summarization:** Long study content can be summarized into concise learning material.
- **Personalized Learning Path:** The system provides learning recommendations based on the learner's needs.
- **Modular API Routes:** Dedicated routes are used for Q&A, explanation, quiz, summarization and recommendations.
- **Gemini Client:** AI communication is organized through a dedicated `gemini_client.py` module.

Main Application Routes

Route	Purpose
/qa	Handles AI-based question answering
/explain	Provides concept explanations
/quiz	Generates topic-based quizzes

Route	Purpose
/summarize	Summarizes educational text
/learn/recommendations	Provides personalized learning recommendations

Setup / Run Instructions

Requirements: Python 3.x, pip, a modern browser, and a Google Gemini API key.

1. Open the project folder in a terminal or VS Code.
2. Create and activate a Python virtual environment.
3. Install the required Python packages using the project's requirements file.
4. Configure the Gemini API key through environment configuration. Do not hard-code the key in source code.
5. Start the FastAPI application using Uvicorn.
6. Open the local address shown by the server in a modern web browser.

Code Quality Checklist

S.No	Criteria	Status
1	Code is modular and organized into suitable modules	Yes
2	Meaningful variable, function and file names are used	Yes
3	Comments/documentation are included where necessary	Yes
4	Input/API error handling is considered	Yes
5	The application runs through the FastAPI server	Yes
6	Project source code can be maintained in version control	Yes

Additional Notes / Comments

- **Modularity:** The project separates the FastAPI application logic, AI interaction through `gemini_client.py`, routes, and frontend templates for easier maintenance.
- **AI Integration:** Google Gemini is used as the core AI service for educational responses, explanations, quiz generation, summarization and learning recommendations.
- **User Experience:** The application is designed to provide multiple learning utilities from a single web application rather than requiring separate tools for each activity.
- **Security:** The Gemini API key should be stored securely as an environment variable and should not be exposed in client-side code or committed to the repository.
- **Future Enhancement:** The project can be extended with user authentication, persistent learner profiles, progress tracking, richer quiz evaluation, analytics and additional AI-powered learning features.